41951- ANÁLISE DE SISTEMAS

Processos de software

Ilídio Oliveira

v2025/04/15

universidade de aveiro
departamento de eletrónica,
telecomunicações e informática

deti

Objetivos de aprendizagem

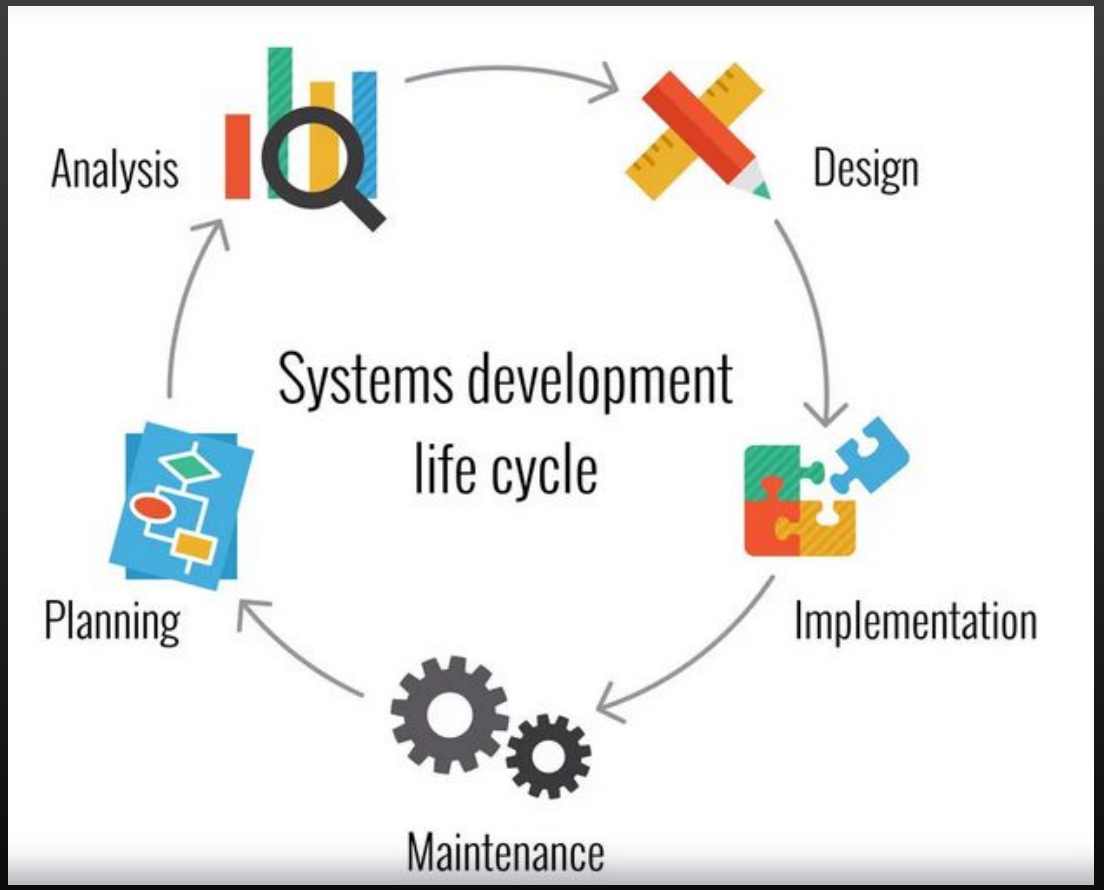
- Identificar atividades comuns a todos os projetos (ciclo de vida)
- Distinguir projetos sequenciais de projetos evolutivos
- Descrever a estrutura do Unified Process (fases, objetivos, iterações)
- Identificar as principais atividades do ciclo de vida
- Mapear disciplinas técnicas nas fases do OpenUP

Fases do SDLC: um modelo geral para a engenharia de software

Alguns autores representam a fase de manutenção

Ideia de evolução do produto

Fechar o "ciclo"



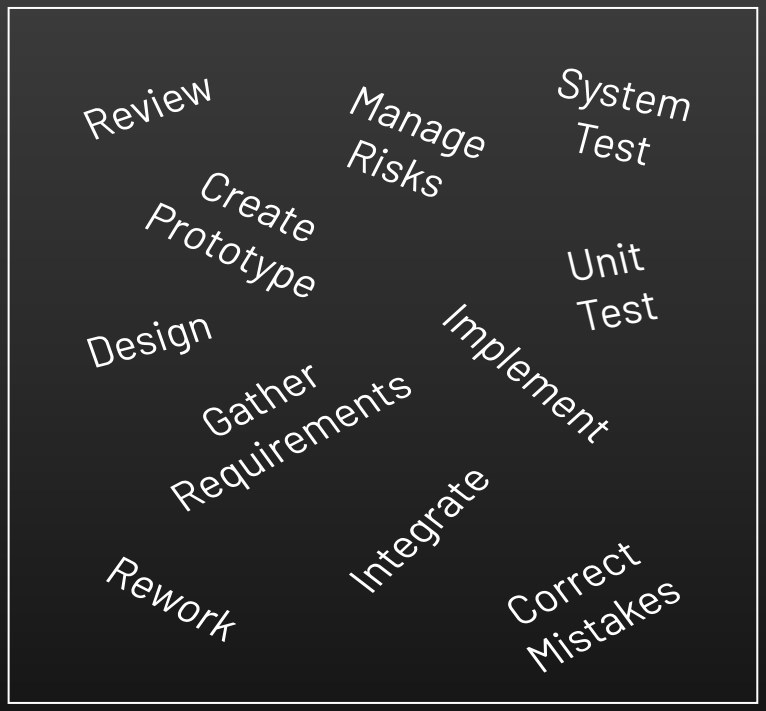
E.g.: Hoffer et al, "Modern Systems Analysis and Design", 5th ed.

Table 1-2 Products of SDLC Phases

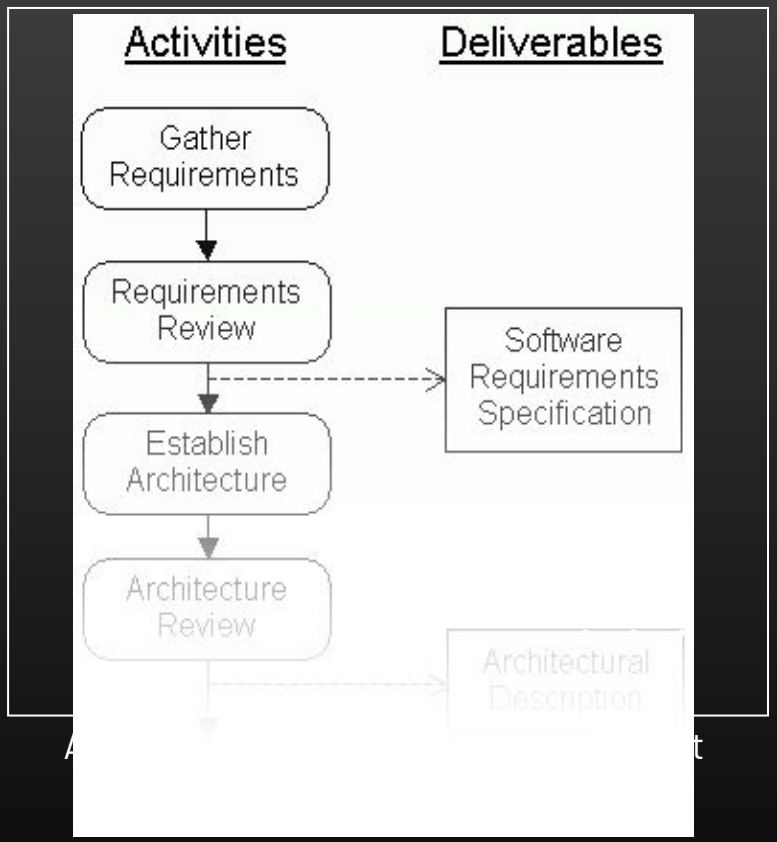
Phase	*Products, Outputs, or Deliverables*
Planning	Priorities for systems and projects; an architecture for data, networks, and selection hardware, and IS management are the result of associated systems;
	Detailed steps, or work plan, for project;
	Specification of system scope and planning and high-level system requirements or features;
	Assignment of team members and other resources;
	System justification or business case
Analysis	Description of current system and where problems or opportunities are with a general recommendation on how to fix, enhance, or replace current system;
	Explanation of alternative systems and justification for chosen alternative
Design	Functional, detailed specifications of all system elements (data, processes, inputs, and outputs);
	Technical, detailed specifications of all system elements (programs, files, network, system software, etc.);
	Acquisition plan for new technology
Implementation	Code, documentation, training procedures, and support capabilities
Maintenance	New versions or releases of software with associated updates to documentation, training, and support

Credit: Hoffer et al, "Modern Systems Analysis and Design", 5th ed.

Como aplicar o SDLC de forma sistemática? □ processo de [desenvolvimento de] software



Developing software without a defined process is chaotic and inefficient



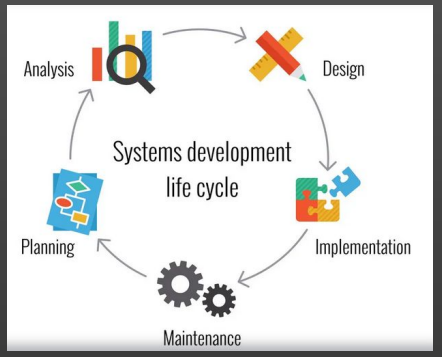
"It is better not to proceed at all, than to proceed without method." — Descartes

Processos de software

Sistematização das atividades, papéis e resultados do processo de desenvolvimento.

Software process

SDLC (visão geral do desenvolvimento)



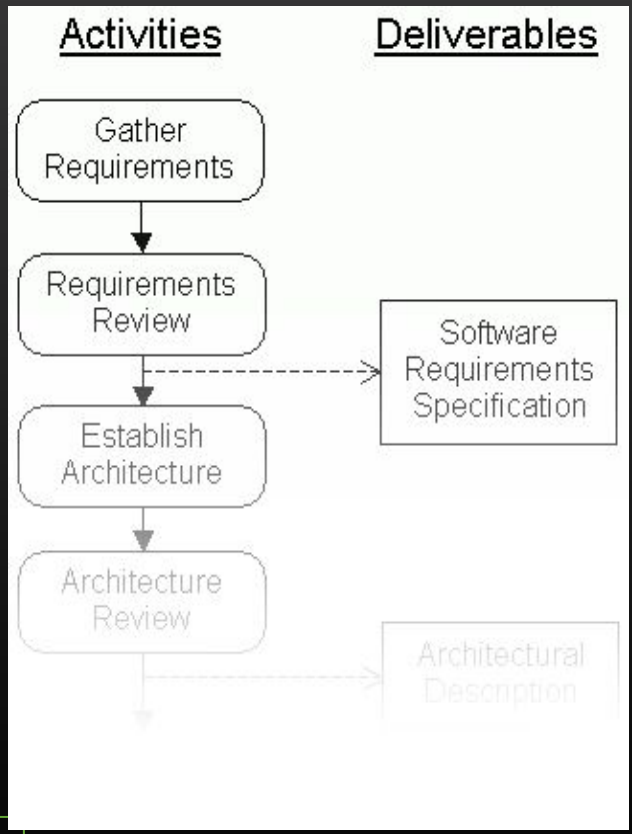
O SDLC é concretizado usando um processo de software sistemático.

Um processo de software é um guião para as atividades e tarefas que são necessárias para construir software de qualidade.

Por que precisamos de um processo explícito?

- As falhas ocorrem frequentemente
- Criar sistemas complexos não é intuitivo
- Os projetos apresentam problemas de implementação (terminados fora do prazo, acima do orçamento ou entregues com menos funcionalidades do que o previsto)

Processo dá o "guião"

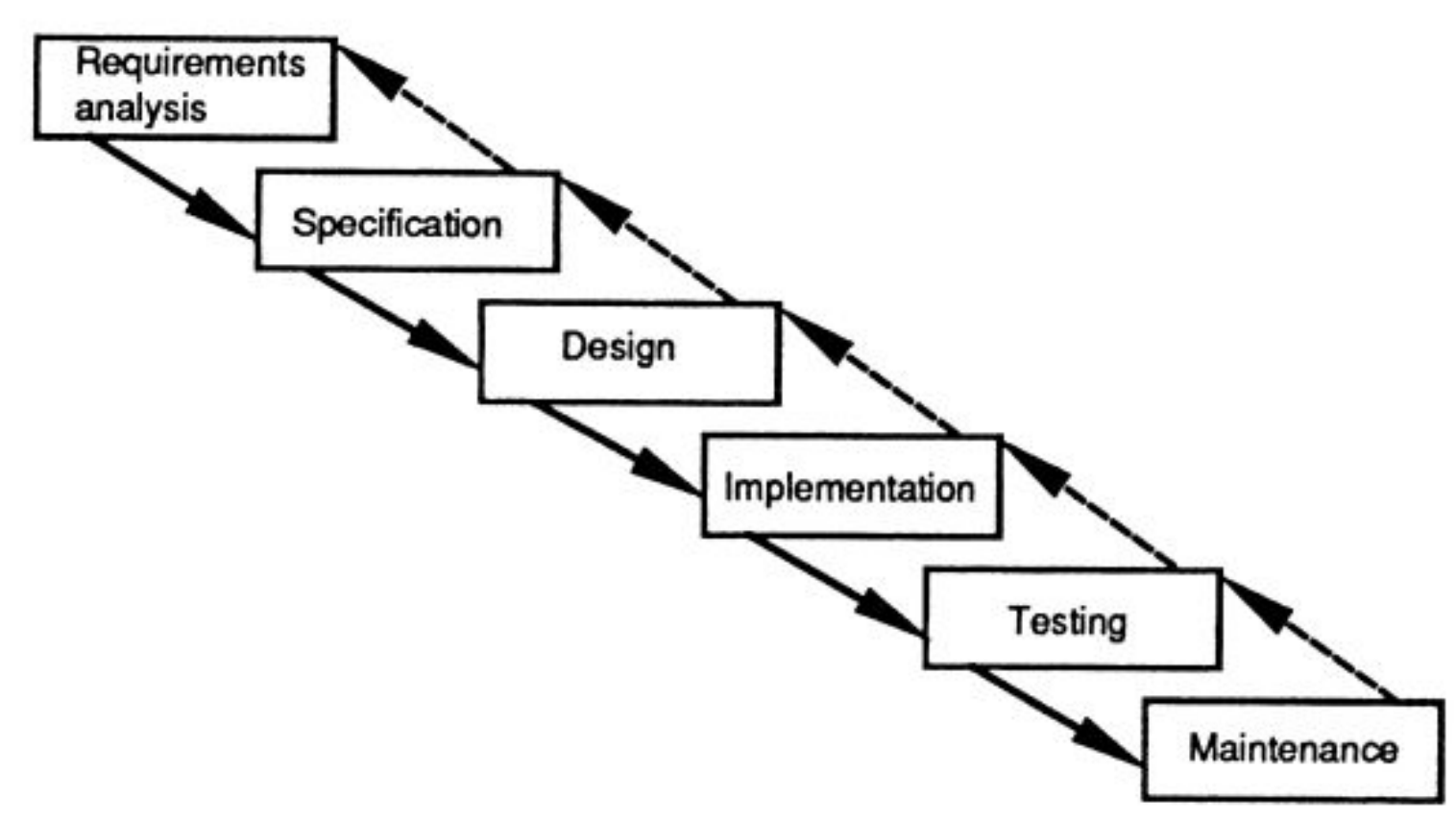


Credit: Dennis et al, "*Systems Analysis and Design: An Object Oriented Approach with UML*", 5th ed.

Metodologias para o desenvolvimento: uma divisão quanto ao ciclo de vida das atividades

Tipo de metodologia	Exemplo
Linear/estruturada	• *Waterfall*
Evolutiva	• *Prototyping* • *Spiral model* • *Métodos ágeis*

“Classic” Waterfall model



W. Royce, “Managing the Development of Large Software Systems,” *Proc. Westcon*, IEEE CS Press, 1970, pp. 328-339.

Waterfall model advantages

Simple and easy to understand and use.

Easy to plan

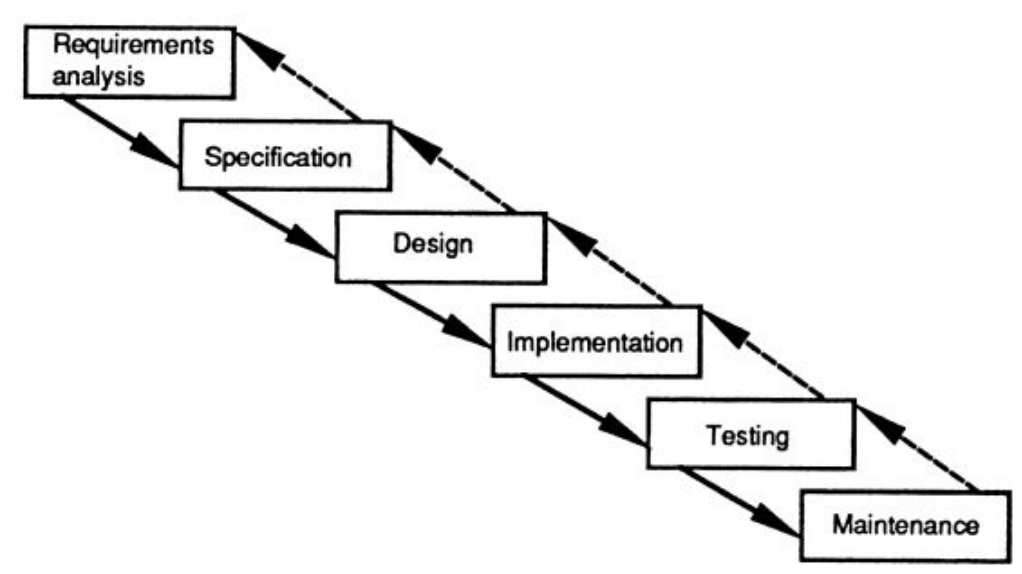
A schedule can be set with deadlines for each stage of development and a product can proceed through the development process like a car in a car-wash, and theoretically, be delivered on time.

Easy to manage

each phase has specific deliverables and a review process.

Phases are processed and completed one at a time.

Works well where requirements are stable and well understood



Waterfall model disadvantages

Problems

Difficulty of accommodating change after the process is underway.

Poor model for long and ongoing projects.

No working software is produced until late during the life cycle.

Not suitable for the projects where requirements are uncertain or at the risk of changing.

Why it may fail?

Real projects rarely follow the sequential flow that the model proposes

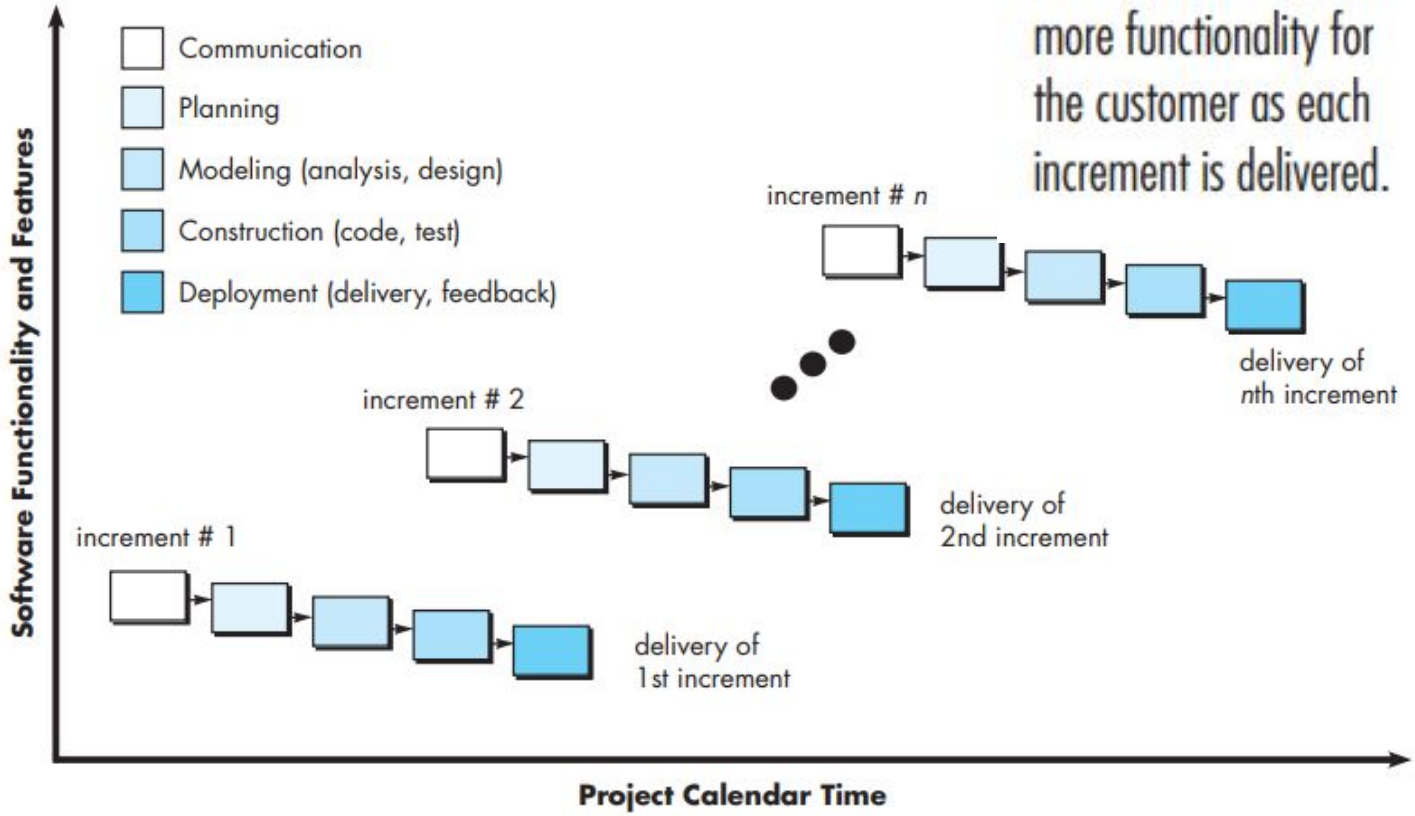
It is often difficult for the customer to state all requirements explicitly

The customer must have patience. A working version of the program(s) will not be available until late in the project time span

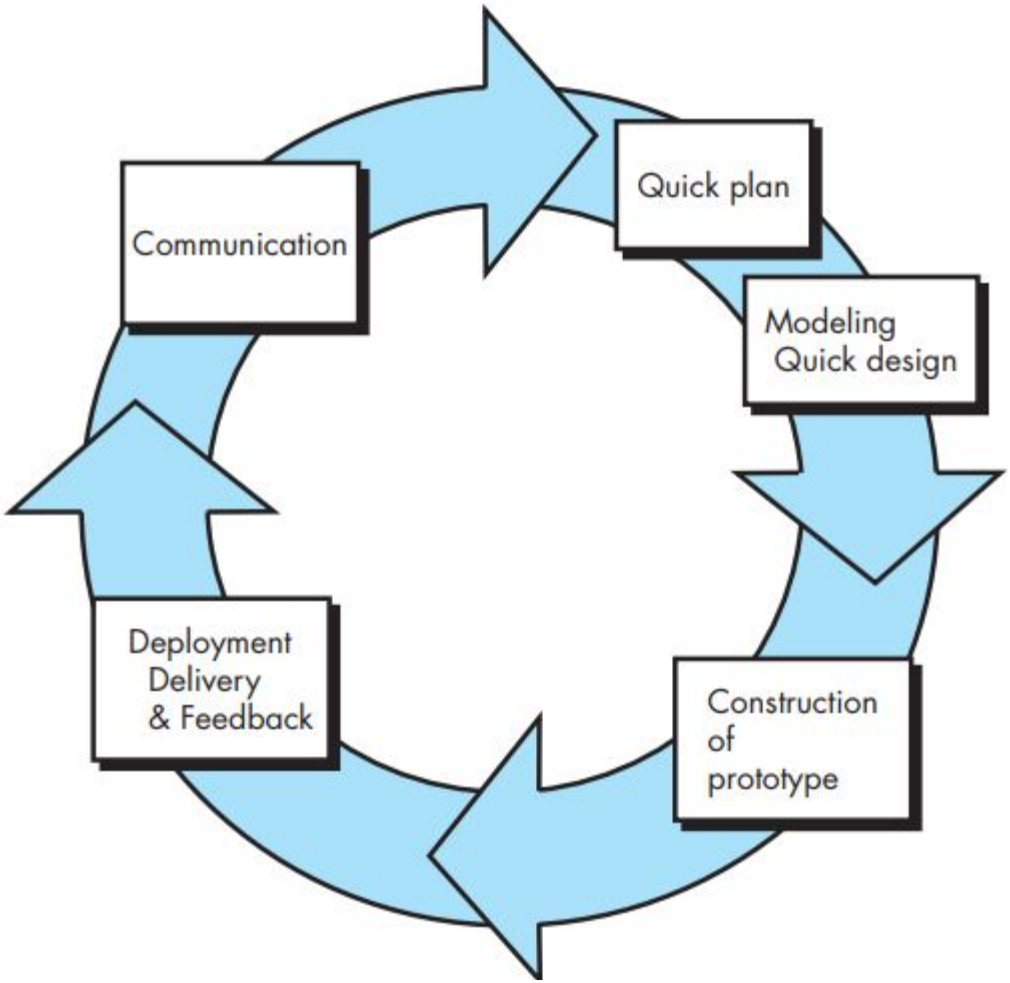
The incremental model delivers a series of releases, called increments, that provide progressively more functionality for the customer as each increment is delivered.

FIGURE 4.3

The incremental model

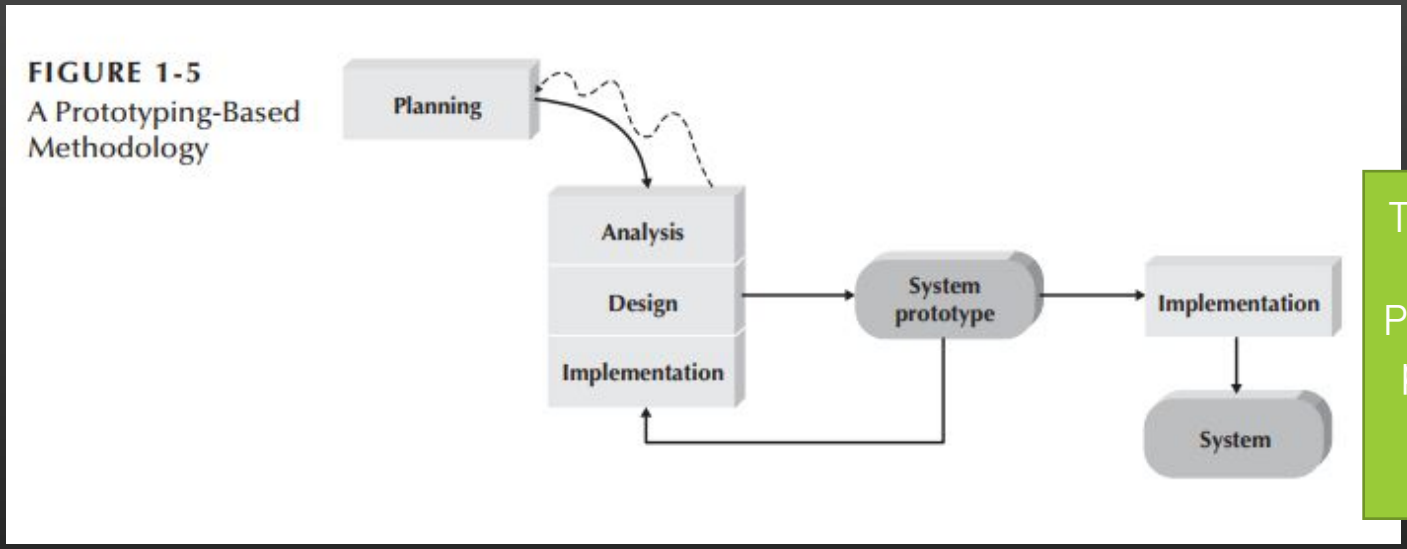


Evolutionary: prototyping

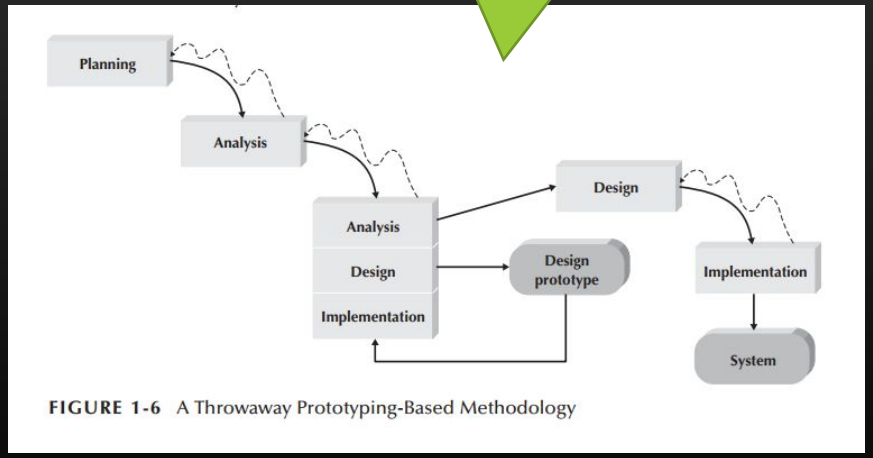


Although problems can occur, prototyping can be an effective paradigm for software engineering. The key is to define the rules of the game at the beginning; that is, all stakeholders should agree that the prototype is built to serve as a mechanism for defining requirements. It is then discarded (at least in part), and the actual software is engineered with an eye toward quality.

RAD: prototyping

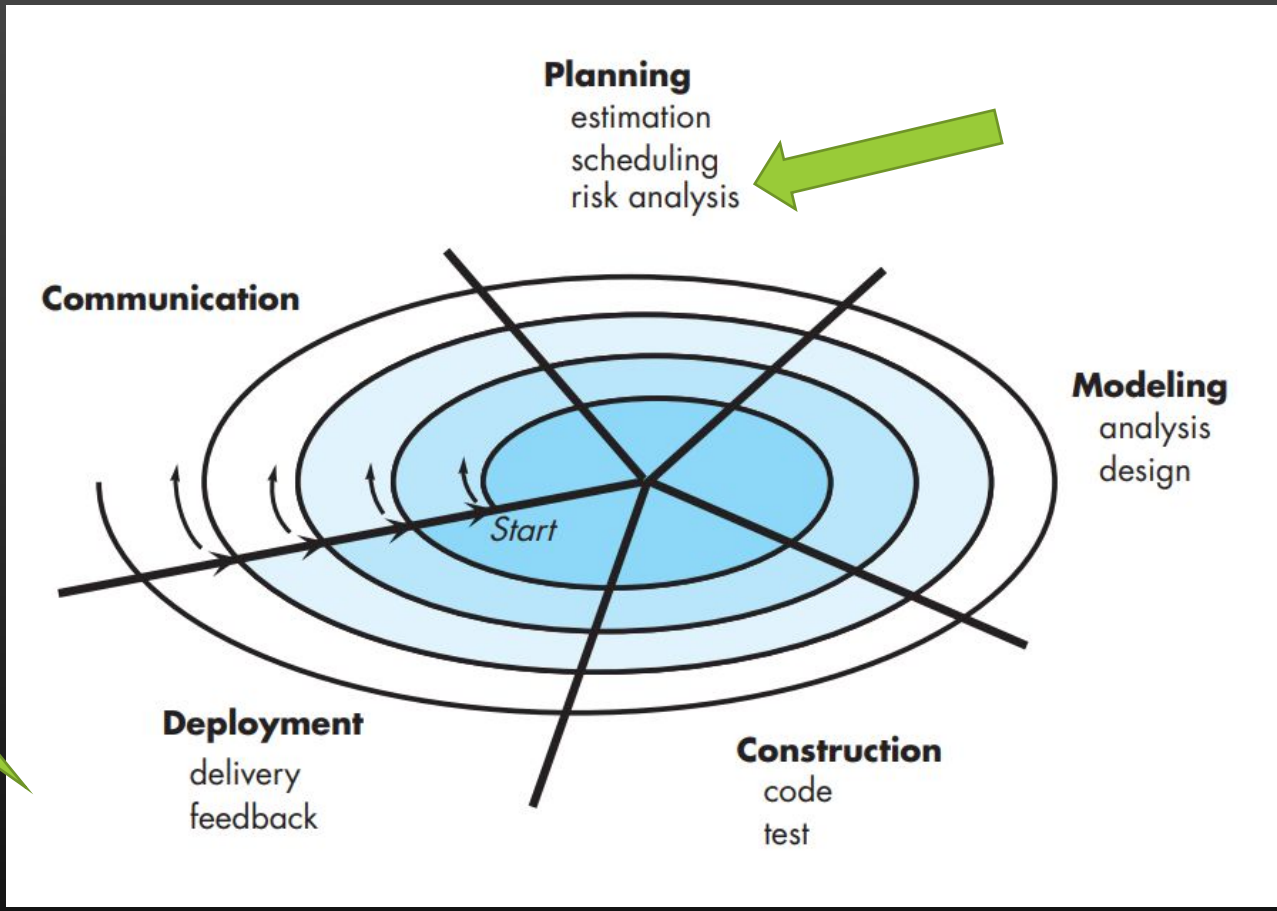


Throwaway prototyping: a variation of Prototyping in which the prototype is discarded (not used for the implementation).



Evolutionary: spiral model

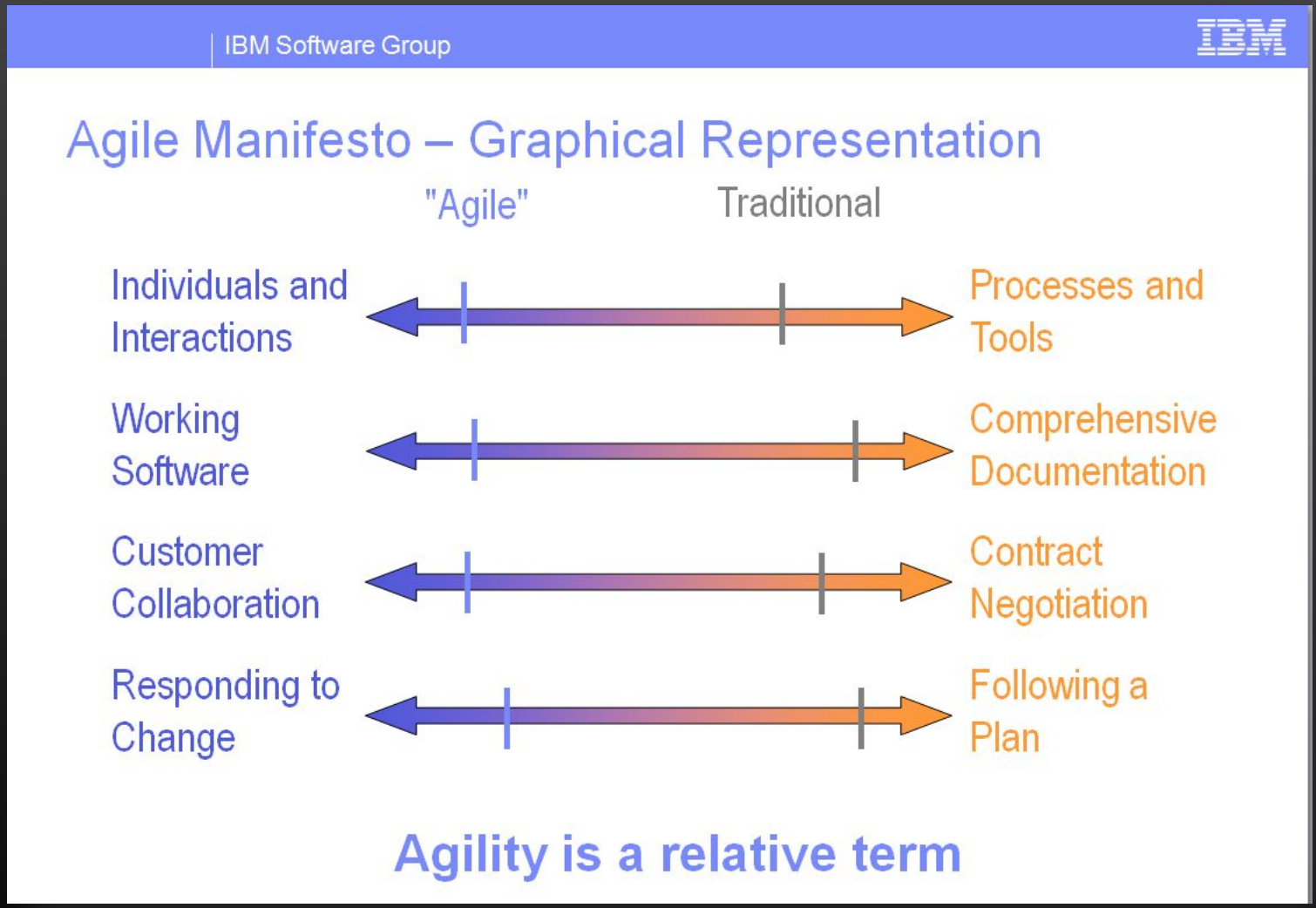
Deployments don't need to be working software



Using the spiral model, software is developed in a series of evolutionary releases. During early iterations, the release might be a model or prototype. During later iterations, increasingly more complete versions of the engineered system are produced.

Agile methods

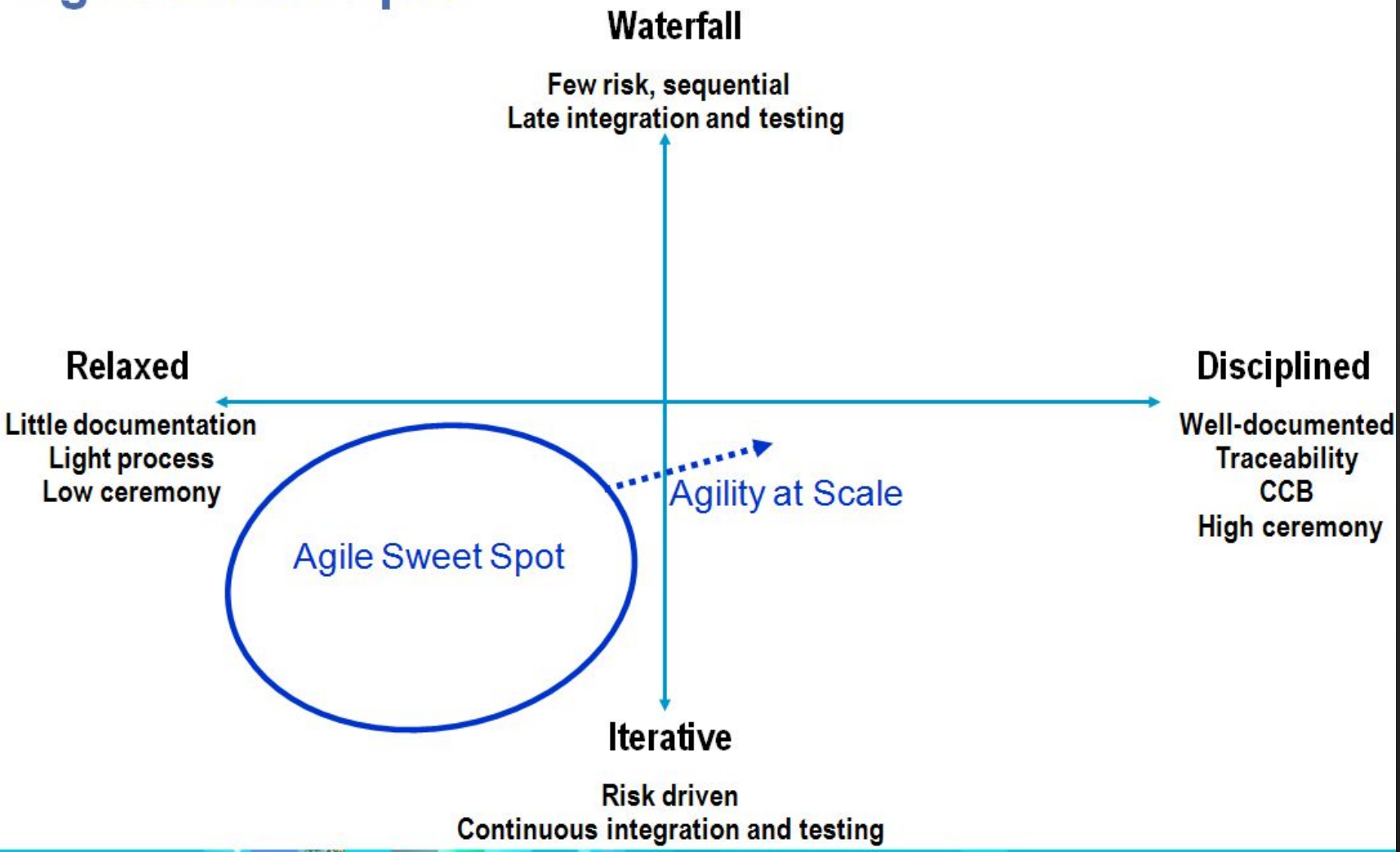
O desenvolvimento ágil de software



http://agilemanifesto.org

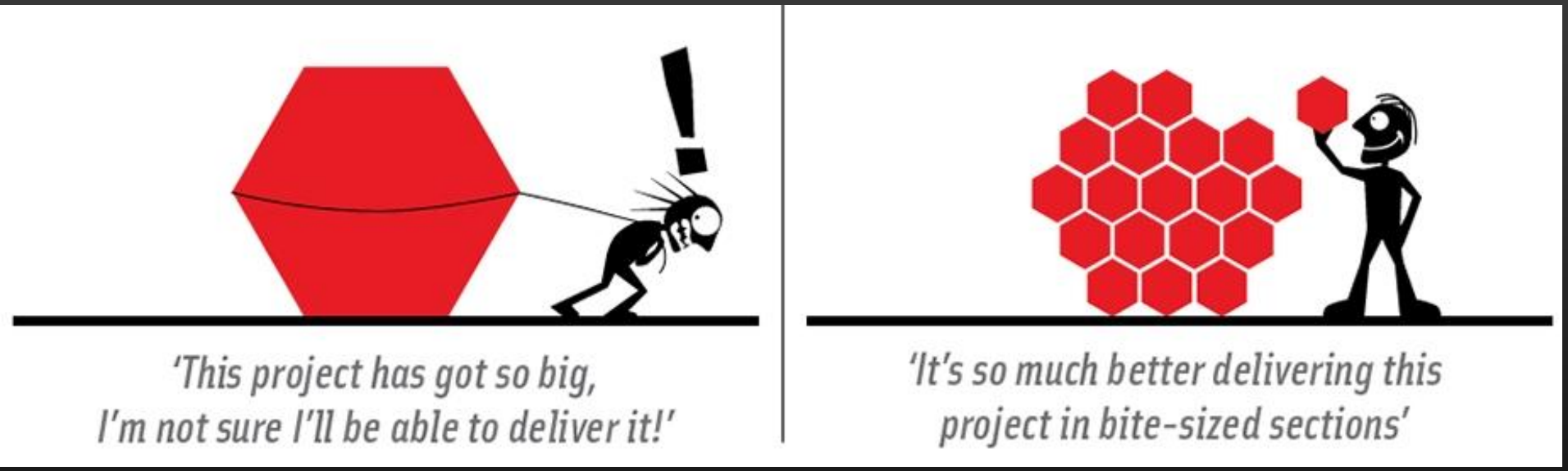
Credit: Per Kroll (IBM)

Agile Sweet Spot

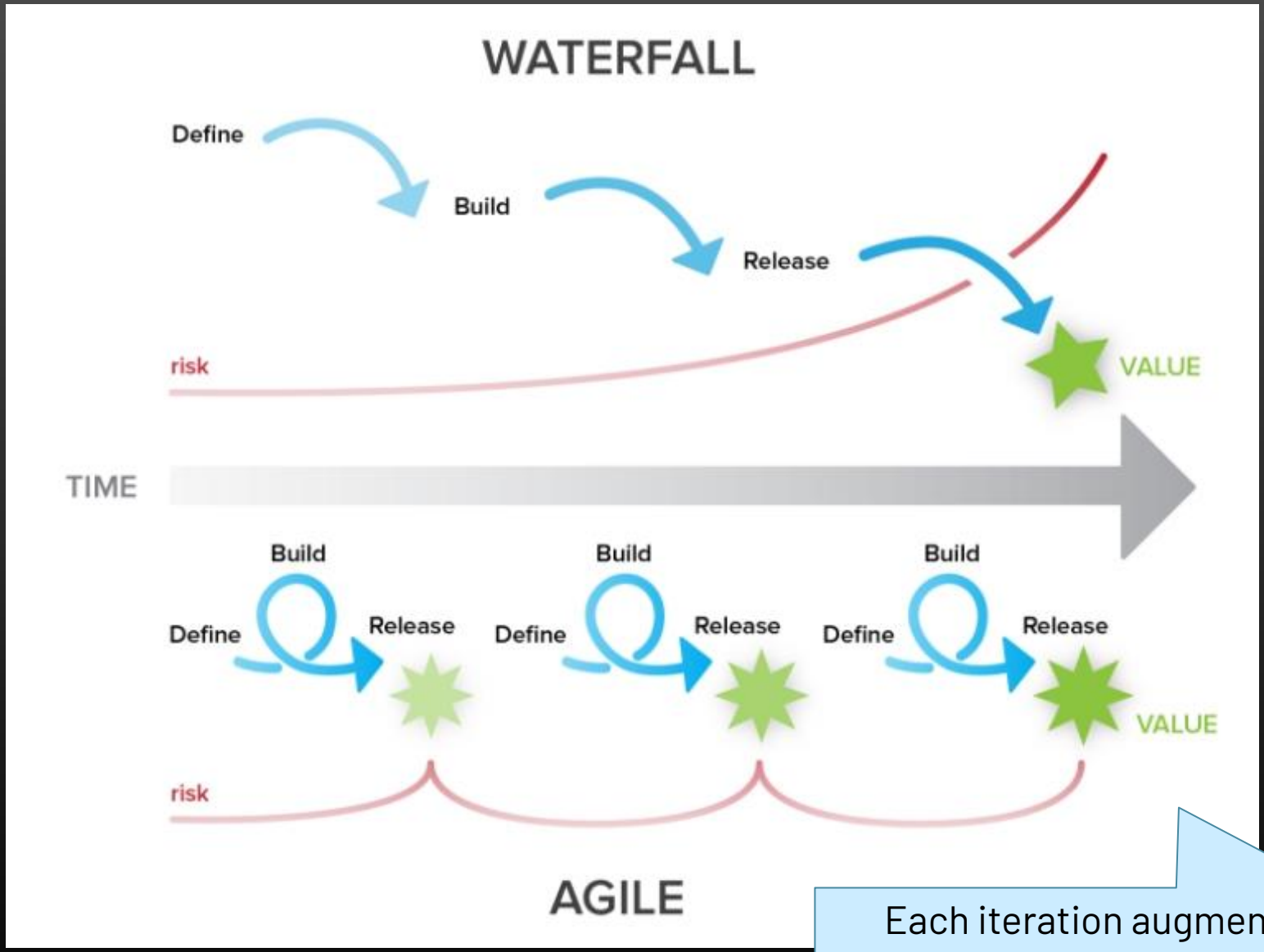


Credit: Per Kroll (IBM)

One project? Micro-projects?



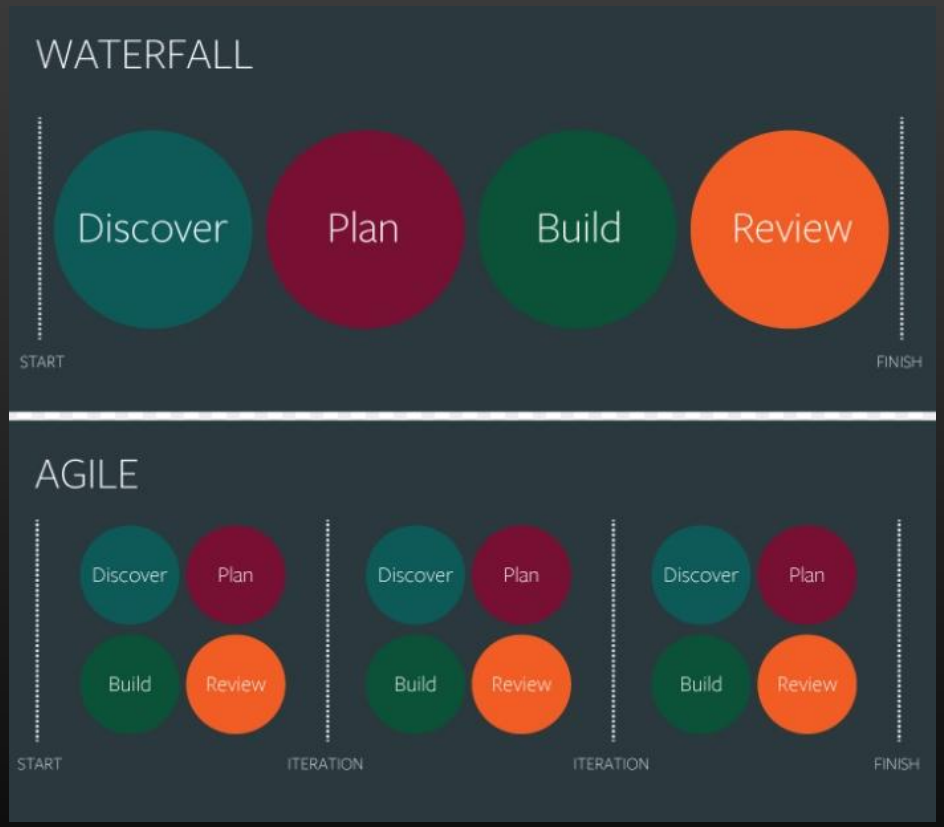
https://blog.ganttpro.com/en/waterfall-vs-agile-with-advantages-and-disadvantages/



https://blog.ganttpro.com/en/waterfall-vs-agile-with-advantages-and-disadvantages/

Each iteration augments the solution by integrating some executable result

To be (agile) or not to be...



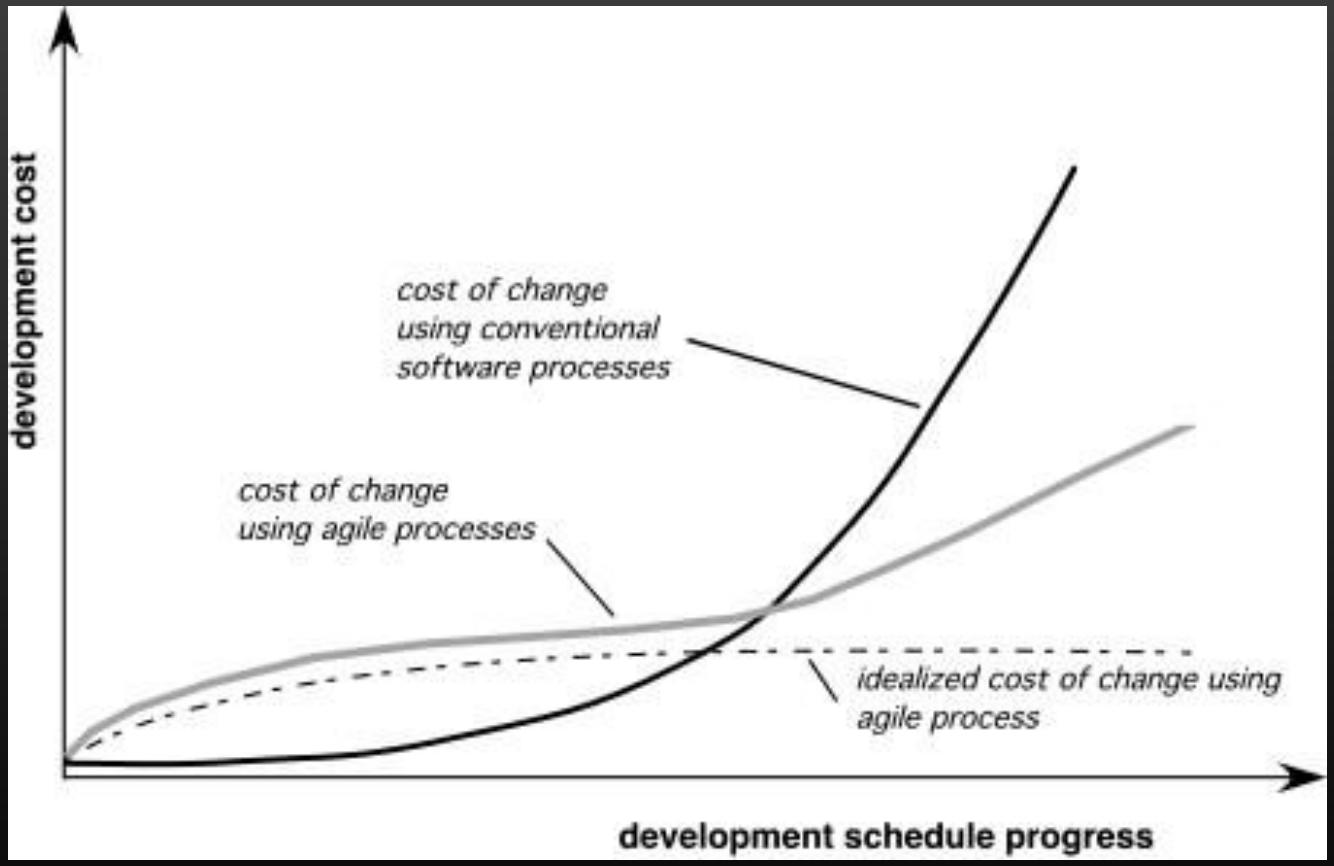
Iterative development (short cycles) *vs* linear development through stages?

Frequent business interaction *vs* fluctuations in stakeholder's participation?

Best to have good collaboration than a detailed plan?

Welcome changes to mitigate risk *vs* avoid changes to control risk?

Agility and the Cost of Change



The cost of change increases nonlinearly as the project progresses

Agility: key points

What is "Agility" in software development?

Effective (rapid and adaptive) response to change

Effective communication among all stakeholders

Drawing the customer onto the team

Organizing a team so that it is in control of the work performed

Yielding ...

Rapid, incremental delivery of software

An Agile Process...

Is driven by customer descriptions of what is required (scenarios)

Recognizes that plans are short-lived

Develops software iteratively with a heavy emphasis on construction activities

Delivers multiple 'software increments'

Adapts as changes occur

Agile & Scrum

Scrum != software process

É um conjunto de práticas para gestão de equipas

Encaixa bem nas ideias de:

- desenvolvimento evolutivo
- Desenvolvimento ágil

Suportada em várias ferramentas

Entidades na gestão de um projeto à moda da "Scrum"

Backlog

- Ordenado por prioridade
- Features valem pontos

Iterações

- *timeboxed*

Iteração atual

- Atribuição e trabalho

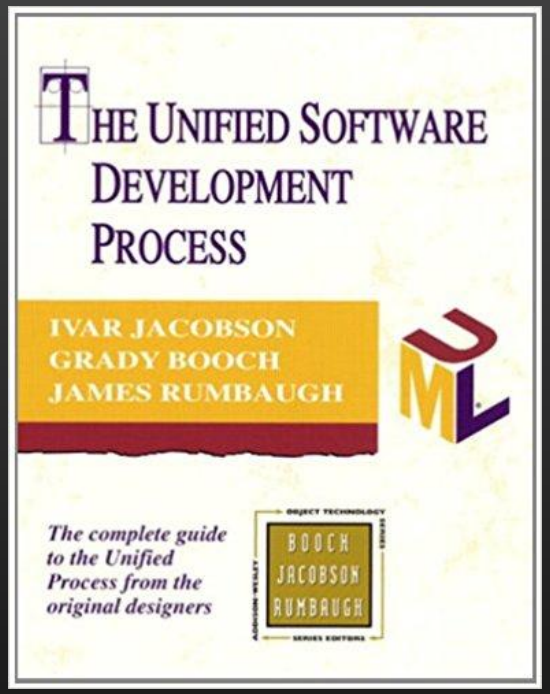
Unified Process/OpenUP

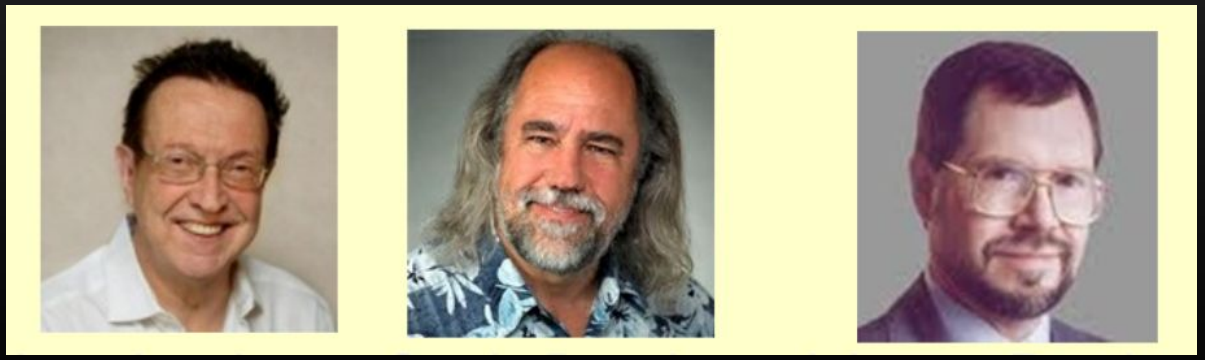
Unified Process/Open Unified Process

Uma tentativa de um processo de genérico

Pode ser adaptado para projetos concretos

O OpenUP é uma versão “livre" do Unified Process





O SDLC é concretizado em processos de desenvolvimento

Um processo especifica:

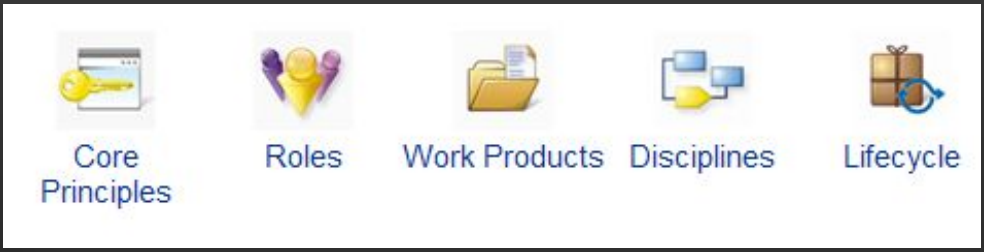
O quê?
Quem?
Como?
Quando?

Um processo inclui:

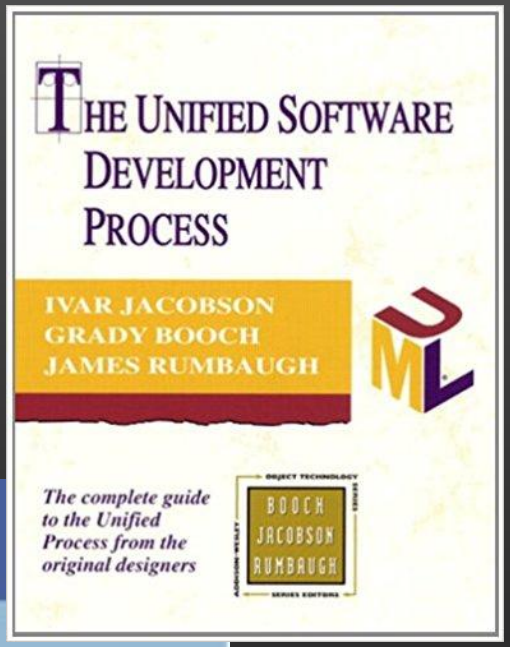
Papéis
Fluxos de trabalho
Procedimentos
Modelos (dos resultados esperados)

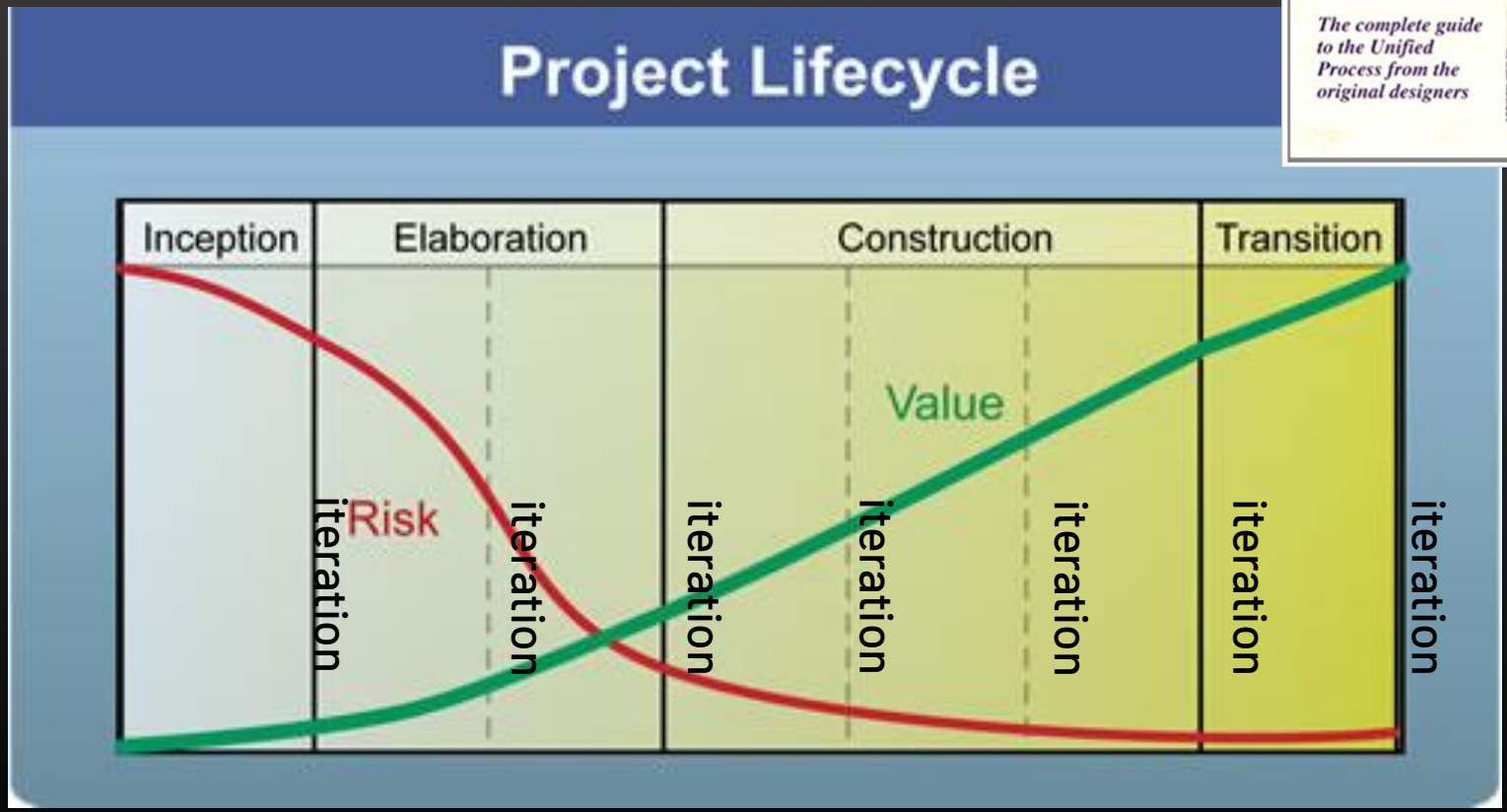
Qual é o melhor processo?

Não há um único "melhor processo"
As organizações devem seleccionar (ou personalizar) o seu processo.

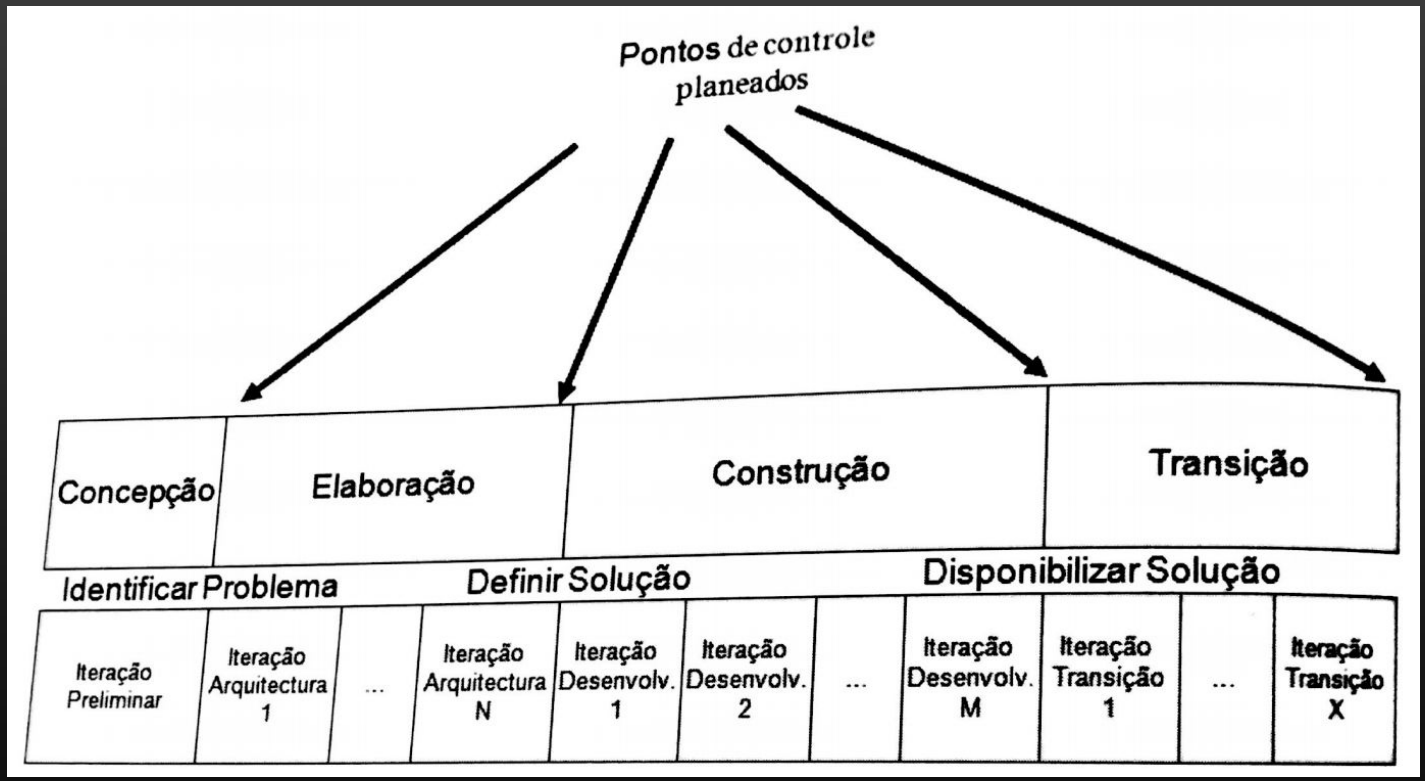


http://sweet.ua.pt/ico/OpenUp/OpenUP_v1514/


THE UNIFIED SOFTWARE DEVELOPMENT PROCESS
IVAR JACOBSON
GRADY BOOCH
JAMES RUMBAUGH
The complete guide to the Unified Process from the original designers
BOOCH
JACOBSON
RUMBAUGH


Project Lifecycle
Inception
Elaboration
Construction
Transition
Risk
Value
iteration
iteration
iteration
iteration
iteration
iteration
iteration

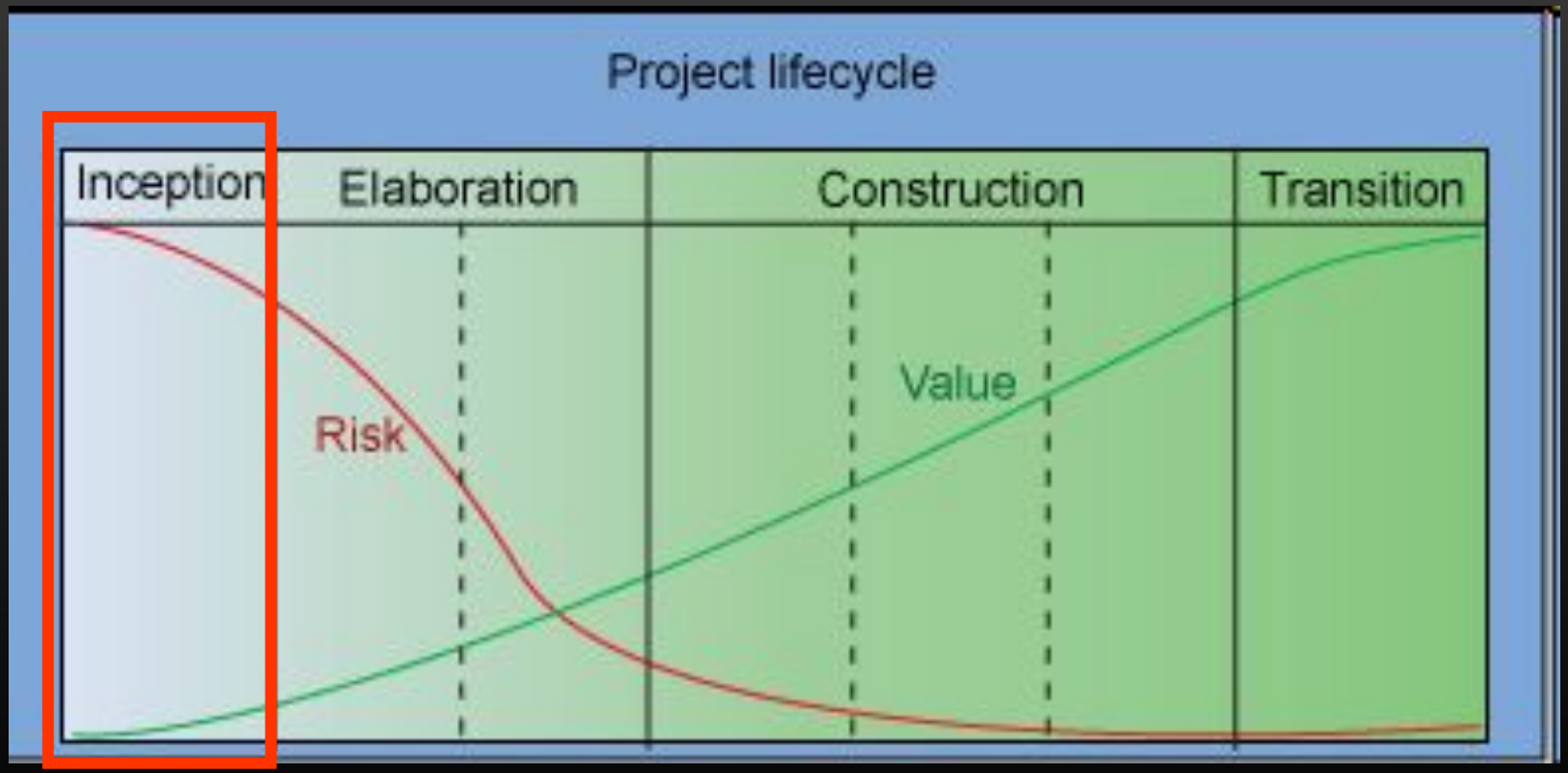
PT: Fases, iterações e pontos de controlo



The phases: Inception

Figura Project lifecycle

Do we agree on project scope and objectives, and whether or not the project should proceed?



Inception: Know What to Build

Typically one short iteration

Produce vision document and initial business case

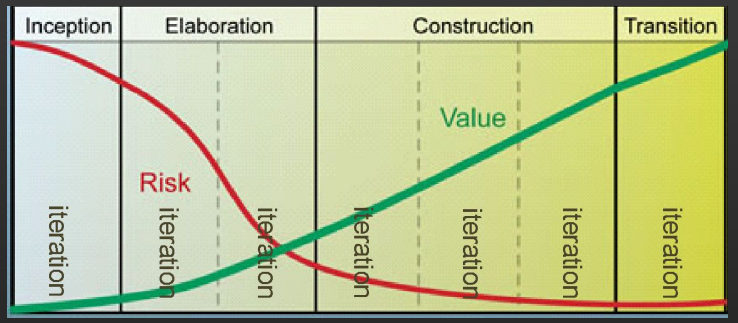
Develop high-level project requirements

- Initial use-case and (optional) domain models (10-20% complete)
- Focus on what is required to get agreement on 'big picture'

Manage project scope

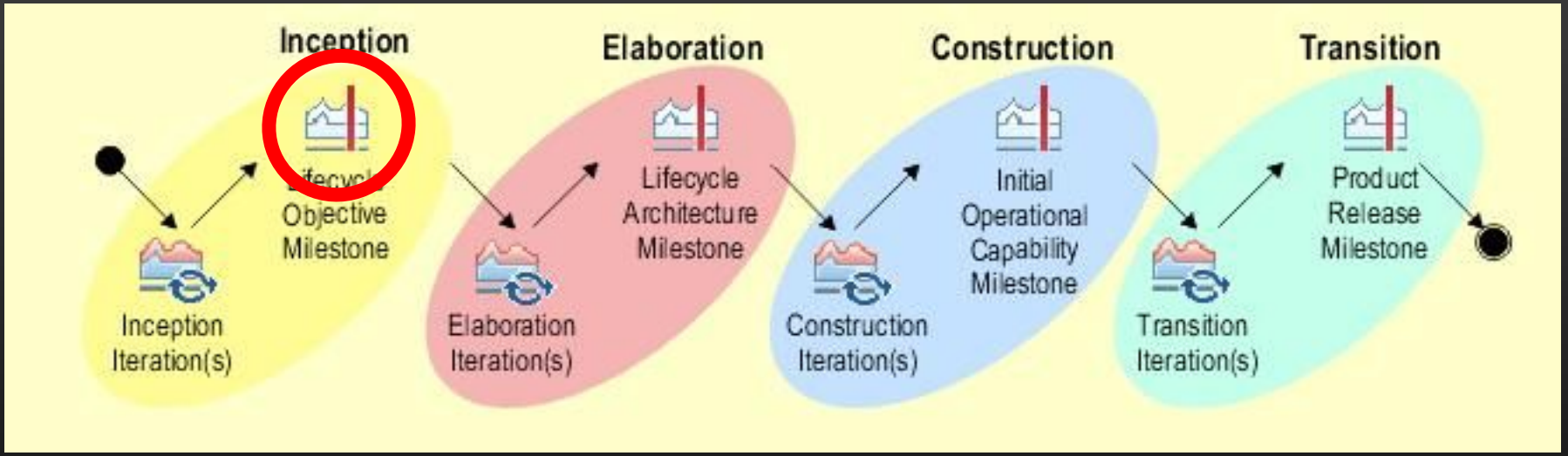
- Reduce risk by identifying key requirements
- Acknowledge that requirements will change
- Manage change, use iterative process

Produce conceptual prototypes as needed



Credit: Per Kroll (IBM)

Milestone: Inception



Objectives Milestone. At this point, you examine the cost versus benefits of the project, and decide either to proceed with the project or to cancel it.

Elaboration: Know How to Build It by Building Some

Elaboration can be a day long or several iterations

Balance

mitigating key technical and business risks with producing value (tested code)

Produce (and validate) an executable architecture

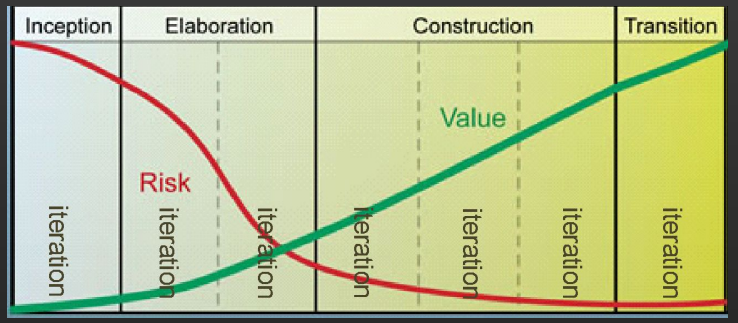
Define, implement and test interfaces of major components. Partially implement some key components.

Identify dependencies on external components and systems. Integrate shells/proxies of them.

Roughly 10% of code is implemented.

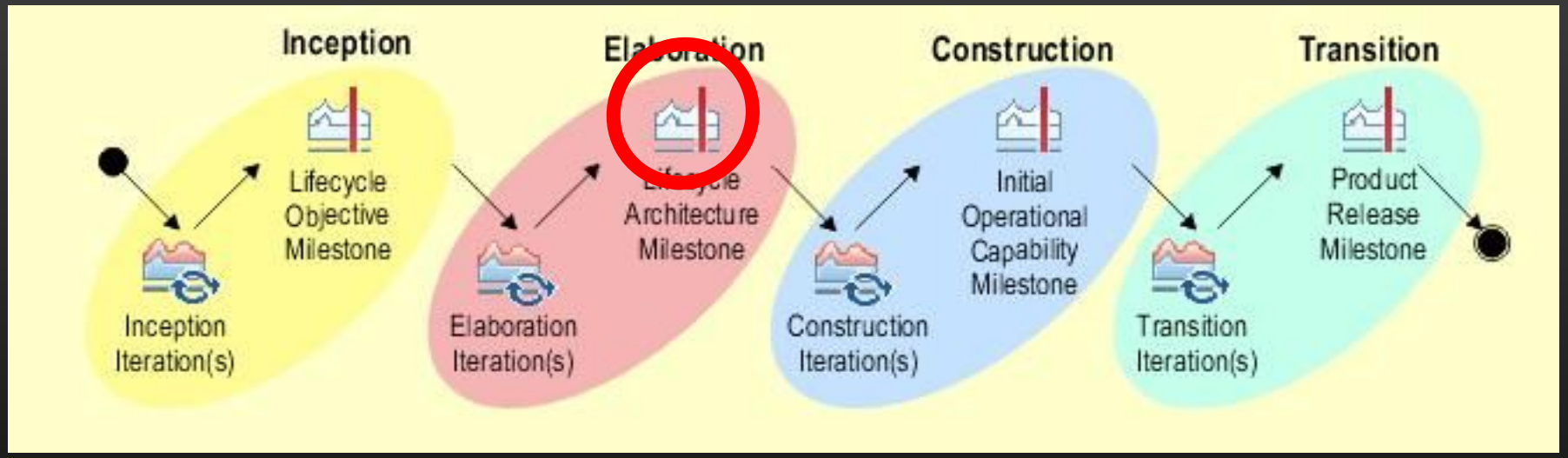
Drive architecture with key use cases

20% of use cases drive 80% of the architecture



Credit: Per Kroll (IBM)

Milestones: Elaboration



Architecture Milestone. At this point, a baseline of requirements is agreed to, you examine the detailed system objectives and scope, the choice of architecture, and the resolution of the major risks. The milestone is achieved when the architecture has been validated.

Construction: Build The Product

Incrementally define, design, implement and test more and more scenarios

- Incrementally evolve executable architecture to complete system
- Evolve architecture as you go along

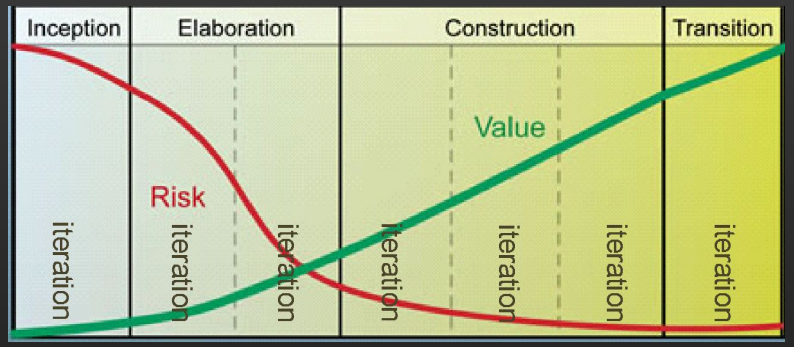
Frequent demonstrations and partial deployment

- Partial deployment strategy depends greatly on what system you build

Daily build with automated build process

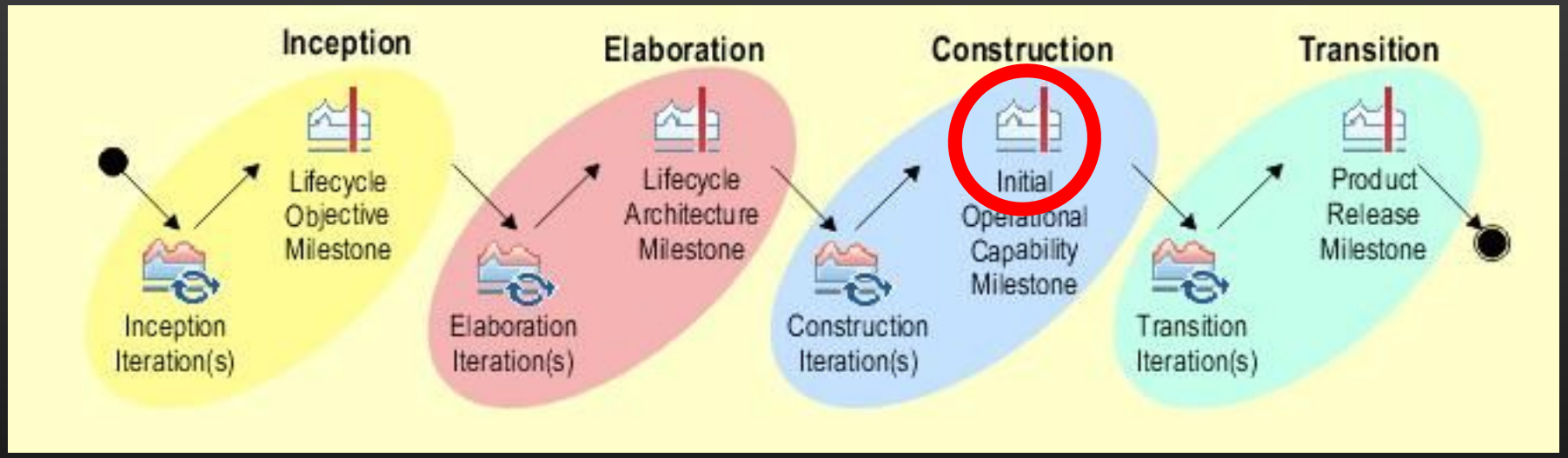
You may have to have a separate test team if you have

- Complex test environments
- Safety or mission critical systems



Credit: Per Kroll (IBM)

Milestones: Construction



Initial Operational Capability Milestone. At this point, the product is ready to be handed over to the transition team. All functionality has been developed and all alpha testing (if any) has been completed. In addition to the software, a user manual has been developed, and there is a description of the current release. The product is ready for beta testing.

Transition: Stabilize and Deploy

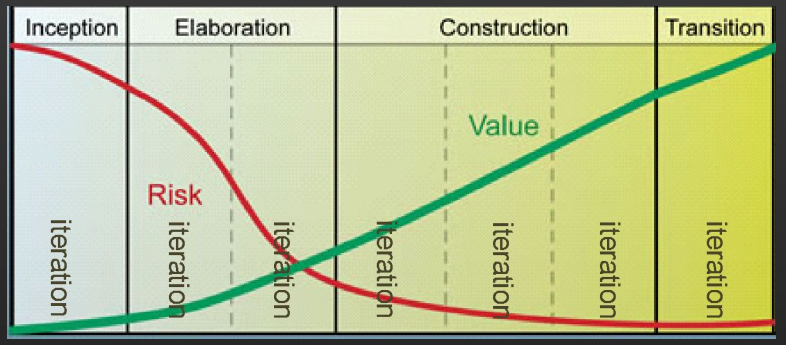
Project moves from focusing on new capabilities to **stabilizing** and tuning

Produce incremental 'bug-fix' releases

Update user manuals and deployment documentation

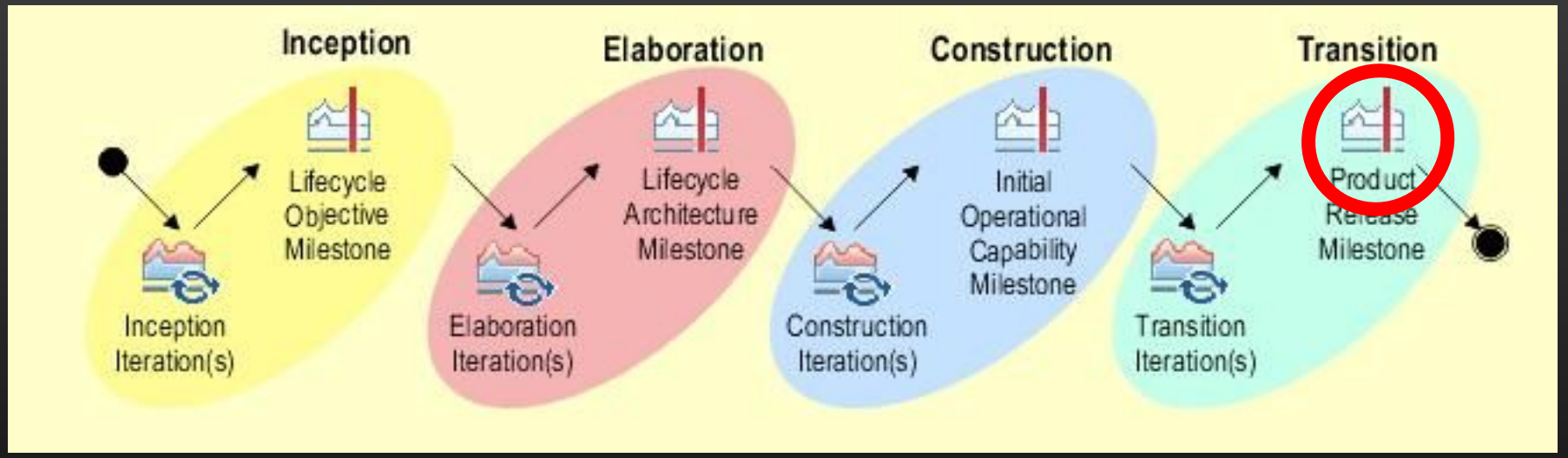
Execute cut-over

Conduct "post-mortem" project analysis



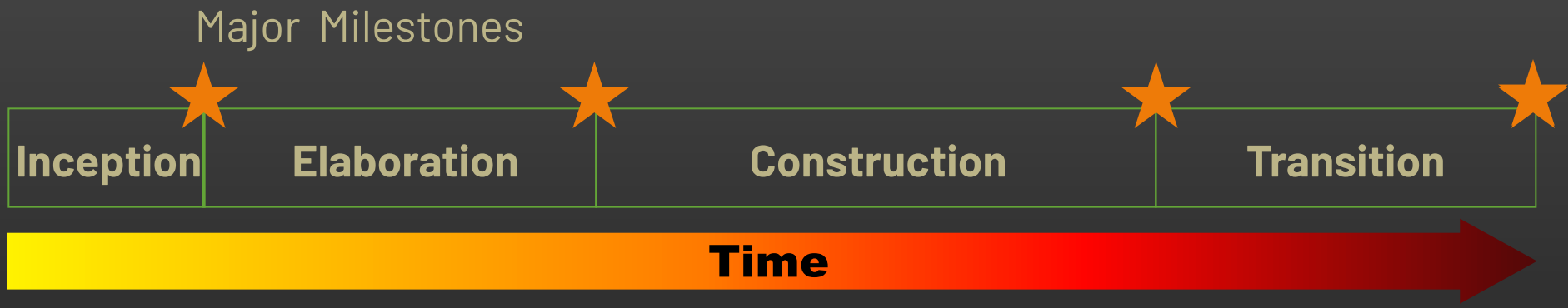
Credit: Per Kroll (IBM)

Milestones: Transition



Product Release Milestone. At this point, you decide if the objectives were met, and if you should start another development cycle. The Product Release Milestone is the result of the customer reviewing and accepting the project deliverables.

Recap main control points (lifecycle milestone)



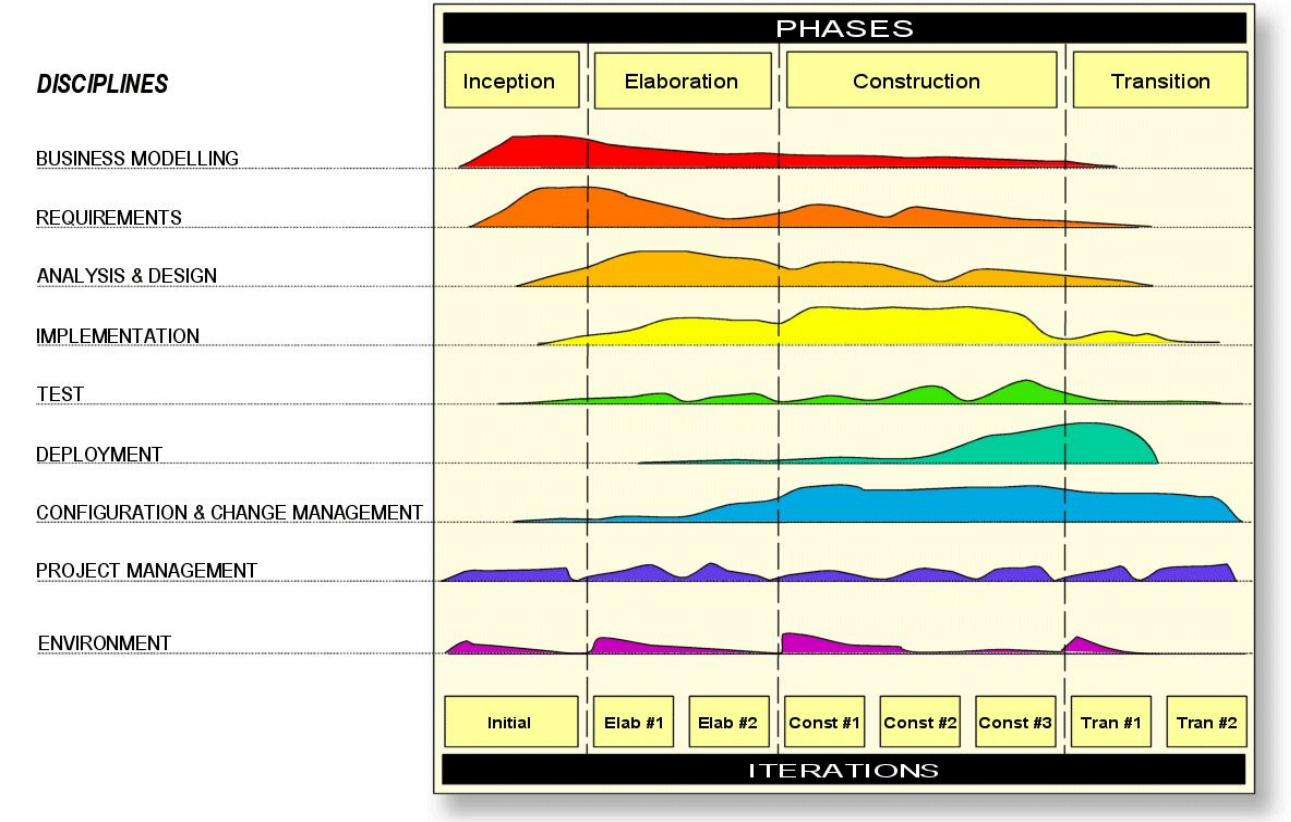
- Inception: Agreement on overall scope
  - Vision, high-level requirements, business case
  - Not detailed requirements
- Elaboration: Agreement on design approach and mitigation of major risks
  - Baseline architecture, key capabilities partially implemented
  - Not detailed design
- Construction: Agreement on complete operational system
  - Develop a beta release with full functionality
- Transition: Validate and deploy solution
  - Stakeholder acceptance, cutover to production

Visão geral do OpenUP/Unified Process

O UP oferece uma abordagem ao SDLC concebida como uma matriz, cruzando diferentes disciplinas técnicas com iterações (evoluções) no projecto.

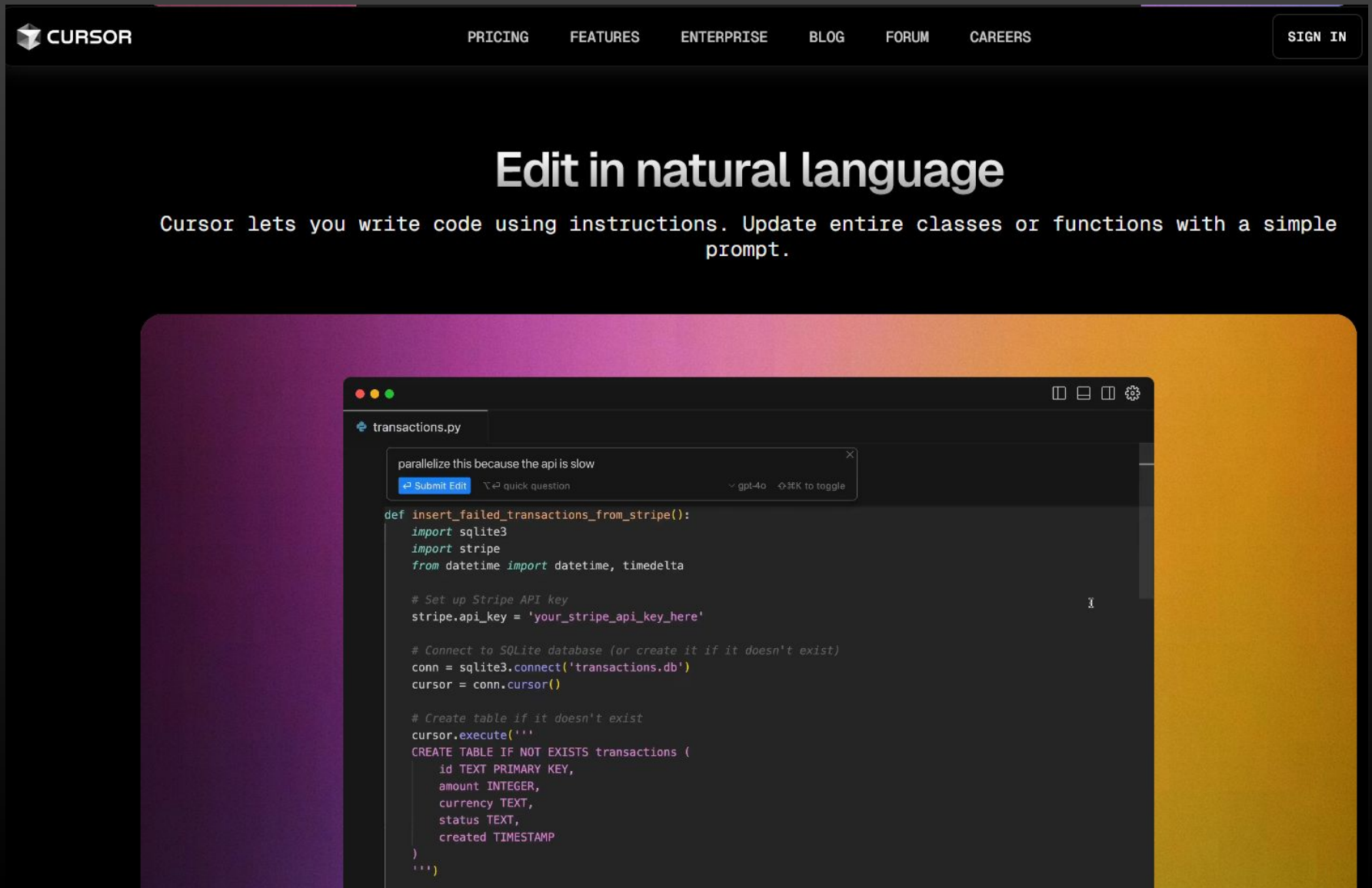
(Nota: fases UP ≠ fases SDLC)

A análise dos requisitos é realizada principalmente no início do projeto (requisitos básicos), mas também durante as iterações (requisitos evolutivos).



Desafios emergentes...

Low-code, genAI,...



Readings & references

Core readings	Suggested readings
• [Pressman'15] – Chap. 4, 5	• [Dennis'15] – Chap 1.